

FIAP

LEONARDO GARROTI

AQUILES FONSECA

LEANDRO DE SOUZA

CHECK POINT 01

FERRAMENTAS OPEN SOURCE DE SEGURANÇA

SÃO PAULO

2026

LEONARDO GARROTI

AQUILES FONSECA

LEANDRO DE SOUZA

CHECK POINT 01

FERRAMENTAS OPEN SOURCE DE SEGURANÇA

Relatório técnico apresentado na disciplina de **CLOUD SECURITY** da FIAP, sob orientação do professor **Fabio Pires**, como parte do Check Point 01.

SÃO PAULO

2026

SUMÁRIO

INTRODUÇÃO E CATEGORIAS	4
PIPELINE, SHIFT-LEFT E SBOM	5
NJSSCAN: IDENTIFICAÇÃO E USO	6
NJSSCAN: RESULTADO REAL	7
NJSSCAN: ANÁLISE DOS ACHADOS	8
NPM AUDIT: IDENTIFICAÇÃO E USO	9
NPM AUDIT: RESULTADO REAL	10
NPM AUDIT: LIMITES E SBOM	11
DATREE: IDENTIFICAÇÃO E FUNCIONAMENTO	12
DATREE: INTEGRAÇÃO E AVALIAÇÃO	13
STACKHAWK: IDENTIFICAÇÃO E FUNCIONAMENTO	14
STACKHAWK: CONFIGURAÇÃO E AVALIAÇÃO	15
LABORATÓRIO E AMBIENTE DOCKER	16
GATE E VERSÃO INSEGURA	17
VERSÃO CORRIGIDA E CONCLUSÃO	18
REFERÊNCIAS	19
ANEXO A - USO DE IA	20

1 INTRODUÇÃO E CATEGORIAS

A segurança de uma aplicação depende do código, das bibliotecas e da configuração do ambiente. Algumas falhas só aparecem quando o sistema está funcionando. Por isso, este trabalho compara ferramentas que verificam partes diferentes de um projeto.

O grupo 3 é formado por Leonardo Garroti, Aquiles Fonseca e Leandro de Souza. As ferramentas indicadas para o grupo são njsscan, npm audit, Datree e StackHawk. Elas representam SAST, SCA, segurança de infraestrutura como código e DAST.

Na parte prática, usamos duas versões de uma aplicação própria em Node.js: uma com falhas intencionais e outra corrigida. A comparação mostra os alertas encontrados, as mudanças no código e a decisão do gate, que bloqueia a entrega quando encontra problemas graves.

Método usado

A pesquisa consultou a documentação e os repositórios dos projetos. O laboratório foi executado em Docker Desktop com WSL 2. O njsscan e o npm audit foram usados na prática. Datree e StackHawk foram estudados, mas não executados.

SAST, SCA, IaC e DAST

Cada categoria tem um objetivo. A tabela apresenta o que ela verifica e em qual condição o teste acontece.

Categoria	O que verifica	Precisa da aplicação rodando?
SAST	Código escrito pela equipe	Não
SCA	Bibliotecas e suas versões	Não
IaC	Arquivos de infraestrutura	Não
DAST	Aplicação funcionando	Sim

SAST e SCA costumam causar confusão. O SAST procura uma falha criada no próprio código. O SCA verifica se uma biblioteca usada pelo projeto possui um aviso de segurança conhecido.

No laboratório, retirar o uso de eval corrige um alerta SAST. Atualizar o lodash corrige o alerta SCA. Uma mudança não substitui a outra.

2 PIPELINE, SHIFT-LEFT E SBOM

Um pipeline executa tarefas em sequência: obter o código, instalar dependências, testar e preparar uma versão. Os testes de segurança podem entrar nessa sequência.

Etapa	Verificação adequada
Code	SAST e IaC
Build	SCA e criação da SBOM
Test	DAST
Deploy	Nova revisão da IaC

Shift-left significa trazer os testes de segurança para o início do desenvolvimento. Assim, a equipe pode corrigir uma falha antes de entregar a aplicação. O DAST completa esse processo ao testar o sistema em funcionamento.

SBOM é uma lista dos componentes usados pelo software. Ela ajuda a responder rapidamente se uma biblioteca com falha está presente. Neste trabalho, o npm gerou arquivos CycloneDX a partir dos lockfiles.

```
npm sbom --sbom-format=cyclonedx --package-lock-only
```

Aplicação no pipeline

O gate transforma a política do grupo em uma decisão automática. Se o njsscan encontra ERROR ou o npm audit encontra HIGH ou CRITICAL, a execução termina com código 1. Um erro de ferramenta ou um relatório inválido termina com código 2. Essa separação evita aprovar uma entrega quando o scanner não conseguiu concluir o teste.

A verificação antecipada reduz o tempo entre a criação e a correção da falha. Mesmo assim, shift-left não significa concentrar todos os testes no início. O DAST continua depois do build porque observa rotas, respostas e configurações que só existem com a aplicação em execução. A SBOM permanece como registro para consultas futuras.

3 NJSSCAN: IDENTIFICAÇÃO E USO

O njsscan é um scanner SAST para aplicações Node.js. Seu mantenedor é Ajin Abraham. A ferramenta é escrita em Python, usa libsast e chama o Semgrep para parte da análise. A licença indicada no projeto é LGPL-3.0.

O repositório consultado foi criado em 2020. A versão usada foi a 1.0.0, publicada em 2026. Na data da pesquisa, o projeto tinha 454 estrelas e 109 forks. Esses números mostram interesse, mas não garantem manutenção contínua.

O njsscan foi escolhido porque trabalha com Node.js, gera JSON, SARIF e HTML e consegue apontar a regra, a linha e o trecho do código. Isso facilita a explicação para quem está aprendendo.

O projeto teve uma atualização recente depois de um período com poucas mudanças. Por isso, uma equipe que usar a ferramenta deve acompanhar novas versões e revisar se as regras continuam atuais.

njsscan: comandos e integração

A instalação pode ser feita com pip. No trabalho, as versões ficam fixadas para que todos usem o mesmo ambiente.

```
pip install njsscan==1.0.0
njsscan app.js --json -o njsscan.json
njsscan app.js --sarif -o njsscan.sarif
```

JSON é simples para ler e contar alertas. SARIF pode ser enviado para plataformas de análise de código. HTML é útil para uma demonstração visual.

O arquivo de configuração permite ignorar caminhos e escolher regras. Uma regra só deve ser ignorada depois de uma revisão e de uma justificativa escrita.

No pipeline preparado, o njsscan roda antes do gate. Um alerta ERROR é tratado como nível HIGH pela regra adotada pelo grupo. Essa regra serve para o laboratório e não cria uma nota CVSS.

4 NJSSCAN: RESULTADO REAL

O njsscan procura padrões perigosos e também usa regras do Semgrep. Ele analisa a estrutura do código sem precisar iniciar a aplicação.

Na versão insegura, o relatório encontrou três problemas:

Regra	Problema	CWE
express_xss	Entrada do usuário em HTML	CWE-79
generic_os_command_exec	Entrada usada em comando	CWE-78
eval_nodejs	Entrada usada em eval	CWE-95

A leitura do código confirmou os três casos. Por isso, eles foram classificados como verdadeiros positivos. A figura apresenta o relatório HTML gerado pelo njsscan no contêiner e permite conferir as regras encontradas.

O scanner não encontra todo tipo de falha. Problemas de login, regra de negócio ou configuração em execução podem exigir outros testes.

Figura 1 - Relatório HTML do njsscan executado no contêiner

```

njsscan: v1.0.0 | Ajin Abraham | opensecurity.in
RULE ID      express_xss
OWASP-WEB    A1: Injection
CWE          CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
DESCRIPTION  Untrusted User Input in Response will result in Reflected Cross Site Scripting Vulnerability.
SEVERITY     ERROR

File         app.js
Match Position 3 - 43
FILES        Line Number(s) 8: 9
Match String  const nome = req.query.nome;
               res.send('<h1>Olá', ' + nome + '</h1>');

RULE ID      generic_os_command_exec
OWASP-WEB    A1: Injection
CWE          CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
DESCRIPTION  User controlled data in 'child_process.exec()' can result in Remote OS Command Execution.
SEVERITY     ERROR

File         app.js
Match Position 3 - 5
FILES        Line Number(s) 12: 14
Match String  exec(req.query.cmd, function (erro, saida) {
               res.type('text').send(erro ? 'Erro' : saida);
               });

RULE ID      eval_nodejs
OWASP-WEB    A1: Injection
CWE          CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code ('Eval Injection')
DESCRIPTION  User controlled data in eval() or similar functions may result in Server Side Injection or Remote Code Injection
SEVERITY     ERROR

File         app.js
Match Position 21 - 41
FILES        Line Number(s) 17
Match String  const resultado = eval(req.query.expr);

```

5 NJSSCAN: ANÁLISE DOS ACHADOS

Os três alertas foram revisados no código. A entrada do usuário chegava ao HTML, ao comando do sistema e ao eval. Como o caminho entre a entrada e a operação perigosa estava visível, os três casos foram classificados como verdadeiros positivos.

Regra	CWE	Correção aplicada
express_xss	CWE-79	Resposta em texto simples
generic_os_command_exec	CWE-78	Retirada da execução de comando
eval_nodejs	CWE-95	Soma de números validados

A amostra teve zero falso positivo entre três alertas, mas ela é pequena e contém falhas intencionais. Esse resultado não pode ser usado para afirmar que a ferramenta terá a mesma precisão em um sistema real. O nível ERROR do njsscan foi tratado como HIGH pelo gate do grupo. Essa equivalência é uma regra do laboratório e não uma nota CVSS.

O alerta de XSS foi resolvido ao responder texto simples. O alerta de comando deixou de existir quando a chamada exec foi retirada. No cálculo, o eval foi trocado por conversão e soma de números. As três mudanças corrigem a causa do problema, sem usar comentários para esconder alertas.

Um resultado do SAST precisa ser lido junto do código. A regra aponta um caminho suspeito, mas a equipe decide se ele pode acontecer e qual mudança preserva a função esperada. Esse cuidado evita aceitar um risco real ou gastar tempo com um alerta sem efeito no projeto.

6 NPM AUDIT: IDENTIFICAÇÃO E USO

O npm audit faz parte do npm CLI. Ele é usado em projetos JavaScript e Node.js para comparar as dependências do projeto com avisos de segurança conhecidos.

O npm CLI é mantido pela equipe do npm dentro da GitHub, empresa da Microsoft. O comando audit foi adicionado em 2018. O código é JavaScript e usa a licença Artistic-2.0.

A versão usada no contêiner foi a 11.12.0. O comando lê o package-lock.json e envia informações das dependências para o serviço de auditoria do registro npm.

Como o comando já faz parte do npm, sua instalação e seu uso são simples. Porém, o resultado depende dos avisos de segurança publicados e de um lockfile que represente as dependências do projeto.

npm audit: comandos, integração e limites

Os comandos principais são curtos:

```
npm audit
npm audit --audit-level=high
npm audit --json
npm audit fix
```

`--audit-level=high` define o nível usado para o código de saída. `--json` cria um resultado que pode ser lido pelo gate. `npm audit fix` tenta atualizar pacotes dentro dos limites permitidos pelo projeto.

O uso de `--force` precisa de cuidado, pois pode instalar uma versão fora da faixa declarada. A equipe deve executar testes depois da atualização.

O npm audit pode entrar no CI logo depois da instalação das dependências. Ele também pode ser usado antes do commit, mas consultas frequentes dependem da rede e do serviço do registro.

Um resultado com zero vulnerabilidades significa apenas que nenhum aviso conhecido foi encontrado nas dependências analisadas naquele momento.

7 NPM AUDIT: RESULTADO REAL

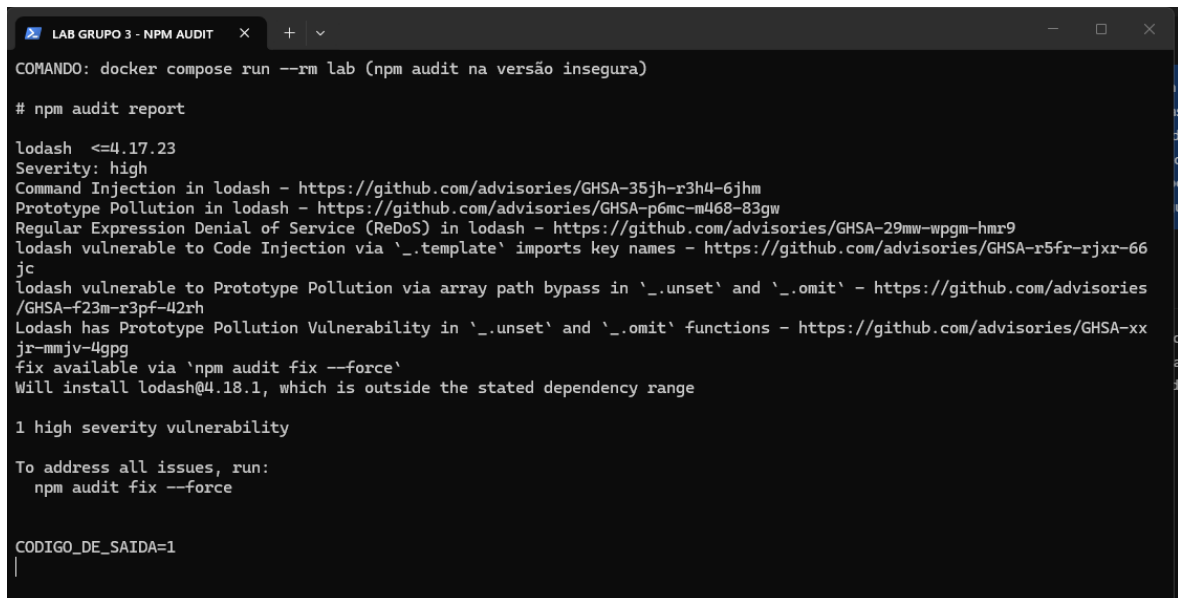
A versão insegura usa lodash 4.17.15. O npm audit encontrou um pacote com severidade HIGH. Esse pacote aparece em vários avisos, mas o resumo conta o pacote vulnerável uma vez.

A presença da versão vulnerável foi confirmada. Isso não prova que todos os avisos podem ser explorados pelas rotas do exemplo. Para responder essa pergunta, seria necessário analisar como cada função da biblioteca é usada.

No gate do laboratório, HIGH e CRITICAL bloqueiam a versão. Depois da atualização para lodash 4.18.1, o comando informou zero vulnerabilidades.

A imagem mostra a saída real do npm audit executado dentro do contêiner na versão insegura.

Figura 2 - npm audit na versão insegura



```
LAB GRUPO 3 - NPM AUDIT x + v
COMANDO: docker compose run --rm lab (npm audit na versão insegura)

# npm audit report

lodash <=4.17.23
Severity: high
Command Injection in lodash - https://github.com/advisories/GHSA-35jh-r3h4-6jhm
Prototype Pollution in lodash - https://github.com/advisories/GHSA-p6mc-m468-83gw
Regular Expression Denial of Service (ReDoS) in lodash - https://github.com/advisories/GHSA-29mw-wpgm-hmr9
lodash vulnerable to Code Injection via `_.template` imports key names - https://github.com/advisories/GHSA-r5fr-rjxr-66jc
lodash vulnerable to Prototype Pollution via array path bypass in `_.unset` and `_.omit` - https://github.com/advisories/GHSA-f23m-r3pf-42rh
lodash has Prototype Pollution Vulnerability in `_.unset` and `_.omit` functions - https://github.com/advisories/GHSA-xxjr-mmjv-4gpg
fix available via `npm audit fix --force`
Will install lodash@4.18.1, which is outside the stated dependency range

1 high severity vulnerability

To address all issues, run:
  npm audit fix --force

CODIGO_DE_SAIDA=1
```

8 NPM AUDIT: LIMITES E SBOM

O npm audit confirmou que o lockfile continha lodash 4.17.15. A presença da versão vulnerável é um verdadeiro positivo de inventário. Mesmo assim, o relatório não prova que todos os avisos ligados ao pacote podem ser explorados pelas rotas do exemplo.

Depois da atualização para lodash 4.18.1, o novo teste informou zero vulnerabilidades conhecidas. Esse resultado vale para a base consultada naquele momento. Novos avisos podem aparecer e exigir outra atualização.

A SBOM complementa o teste porque registra os componentes e as versões presentes no projeto. Neste trabalho, o npm produziu arquivos CycloneDX a partir dos lockfiles. O documento ajuda a responder se uma biblioteca afetada está presente, mas não substitui a análise do código nem o teste da aplicação.

O comando npm audit fix pode resolver atualizações compatíveis. O uso de --force exige mais cuidado, pois pode instalar uma versão fora da faixa esperada pelo projeto. Depois de qualquer atualização, a equipe deve repetir os testes da aplicação e conferir se o comportamento foi mantido.

Um falso negativo pode acontecer quando o lockfile está incompleto ou quando a falha ainda não foi publicada. Um alerta também pode ter pouco efeito prático quando a função vulnerável não é usada. Por isso, o inventário confirma a presença do componente, enquanto a revisão do uso confirma o risco para a aplicação.

9 DATREE: IDENTIFICAÇÃO E FUNCIONAMENTO

Datree foi uma ferramenta de segurança para arquivos Kubernetes. O CLI era escrito em Go e distribuído com licença Apache-2.0. Ele avaliava arquivos YAML antes de chegarem ao cluster.

O projeto foi arquivado no GitHub em outubro de 2024. A própria empresa informou o fim da manutenção em 2023. A última versão encontrada foi a 1.9.19.

A falta de manutenção limita sua adoção. O Datree ainda serve para estudar políticas de Kubernetes, mas um projeto novo deve considerar uma ferramenta que continue recebendo atualizações.

O enunciado atribui Datree ao grupo, por isso o nome foi mantido na pesquisa. Para um uso real atual, seria melhor comparar alternativas mantidas, como Checkov, Trivy ou Kubescape.

Datree: funcionamento e exemplo de IaC

O Datree lê um arquivo Kubernetes, verifica se o YAML é válido e aplica regras de política. Uma regra pode avisar sobre privilégio alto, ausência de limites de recursos ou uso inseguro de imagem.

No exemplo inseguro do trabalho, o contêiner usa privilégio elevado e não define limites. Na versão corrigida, o privilégio é retirado, o sistema de arquivos fica somente para leitura e os recursos recebem limites.

```
datree test iac/pod-inseguro.yaml --schema-version 1.20.0
```

A análise acontece antes do deploy. Isso reduz a chance de uma configuração ruim chegar ao cluster.

A ferramenta não verifica o comportamento da aplicação, a segurança da imagem ou as permissões reais do ambiente. Esses pontos precisam de outras verificações.

10 DATREE: INTEGRAÇÃO E AVALIAÇÃO

O CLI podia ser instalado por script, pacote ou contêiner. Também podia rodar em GitHub Actions, GitLab CI e outros sistemas de CI.

```
datree test iac/pod-inseguro.yaml --output json
```

Uma equipe podia criar políticas próprias e definir quais regras bloqueariam o pipeline. O modo offline permitia usar regras guardadas localmente.

Neste trabalho, o Datree não foi executado porque o projeto está arquivado. Foram mantidos apenas os arquivos de exemplo e a explicação do funcionamento.

Para adotar uma alternativa, o grupo deveria repetir o mesmo processo do laboratório: fixar versão, executar contra um exemplo conhecido, guardar o relatório e definir um gate claro.

Avaliação para uso atual

O principal limite do Datree é a falta de manutenção. Uma ferramenta de segurança precisa acompanhar mudanças do Kubernetes e novas formas de configuração. Usar um projeto arquivado pode gerar regras antigas, problemas de instalação e falta de correção para erros do próprio scanner.

Em uma escolha atual, a equipe deveria comparar alternativas mantidas, verificar a licença, fixar uma versão e testar arquivos com resultado conhecido. Também seria necessário registrar a saída em JSON ou SARIF e definir quais políticas bloqueiam o pipeline. O exemplo deste trabalho permite repetir essa avaliação sem afirmar que o Datree foi executado.

11 STACKHAWK: IDENTIFICAÇÃO E FUNCIONAMENTO

StackHawk é uma plataforma de testes DAST. Seu scanner é chamado HawkScan e utiliza uma base ligada ao OWASP ZAP. O uso comum acontece por contêiner.

A empresa StackHawk mantém a plataforma e a documentação. O serviço possui partes comerciais. Isso precisa ser explicado porque o trabalho pede ferramentas open source. A base do scanner tem origem aberta, mas a experiência completa depende da plataforma.

A versão 6.0.0 do HawkScan foi anunciada em 2026. O produto continua ativo. Mesmo assim, a equipe deve conferir plano, limites e necessidade de conta antes de usar em um projeto.

O StackHawk foi mantido na pesquisa por estar na lista do grupo. Ele não foi usado no laboratório obrigatório.

StackHawk: como o DAST funciona

O DAST testa a aplicação funcionando. Ele descobre rotas, envia entradas de teste e observa respostas. Isso permite encontrar problemas que dependem do servidor e da configuração do ambiente.

O fluxo básico é:

Passo	Ação
1	Iniciar a aplicação de teste
2	Definir o endereço permitido
3	Executar o scanner
4	Revisar os alertas
5	Corrigir e testar novamente

A autenticação exige configuração extra quando as rotas são protegidas. Sem isso, o scanner pode enxergar apenas a tela de login.

DAST pode gerar falso positivo e também deixar rotas sem teste. Ele deve ser executado somente contra um alvo próprio ou autorizado.

12 STACKHAWK: CONFIGURAÇÃO E AVALIAÇÃO

A configuração usa um arquivo YAML com o endereço da aplicação, o identificador do projeto e opções de ambiente. Segredos devem ficar nas variáveis protegidas do CI.

```
docker run --rm -v "$PWD:/hawk" stackhawk/hawkscan:latest
```

Em uma equipe, é melhor fixar a versão da imagem em vez de usar `latest`. Isso evita que uma atualização mude o resultado sem aviso.

O scanner pode rodar depois que o ambiente de teste estiver disponível. O relatório precisa ser guardado como artefato. Antes de bloquear o build, a equipe deve revisar quais níveis são confiáveis para o projeto.

StackHawk não substitui SAST ou SCA. Ele observa o sistema por fora e não conhece todo o caminho interno do código.

Cuidados na execução

O endereço do alvo deve apontar apenas para um ambiente autorizado. O scanner também precisa de limites de tempo e de requisições para não atrapalhar outros testes. Quando existe login, a configuração deve permitir que o HawkScan alcance as rotas protegidas. Sem isso, o relatório pode dar uma falsa impressão de cobertura.

Os alertas do DAST precisam de revisão porque uma resposta incomum pode não representar uma falha explorável. A equipe deve guardar o relatório, reproduzir o caso e repetir o teste após a correção. Para o trabalho do grupo 3, essa execução ficou como planejamento; nenhum número ou resultado do StackHawk foi inventado.

13 LABORATÓRIO E AMBIENTE DOCKER

O laboratório usa uma aplicação Node.js criada para a atividade. Há um alvo inseguro e outro corrigido. Os dois ficam na mesma imagem para facilitar a comparação.

O Dockerfile instala Node.js, Python, njsscan, Semgrep e npm. O Docker Compose executa o script de análise e grava os relatórios. A imagem criada recebeu o nome `cp01-grupo3:1.0`.

A construção e as duas execuções foram realizadas no Docker Desktop com WSL 2. A tela do Docker Desktop confirma a imagem local. A tela do terminal registra as versões e o identificador da imagem.

Esse ambiente evita que cada colega precise instalar todas as ferramentas diretamente no Windows.

Figura 3 - Imagem no Docker Desktop

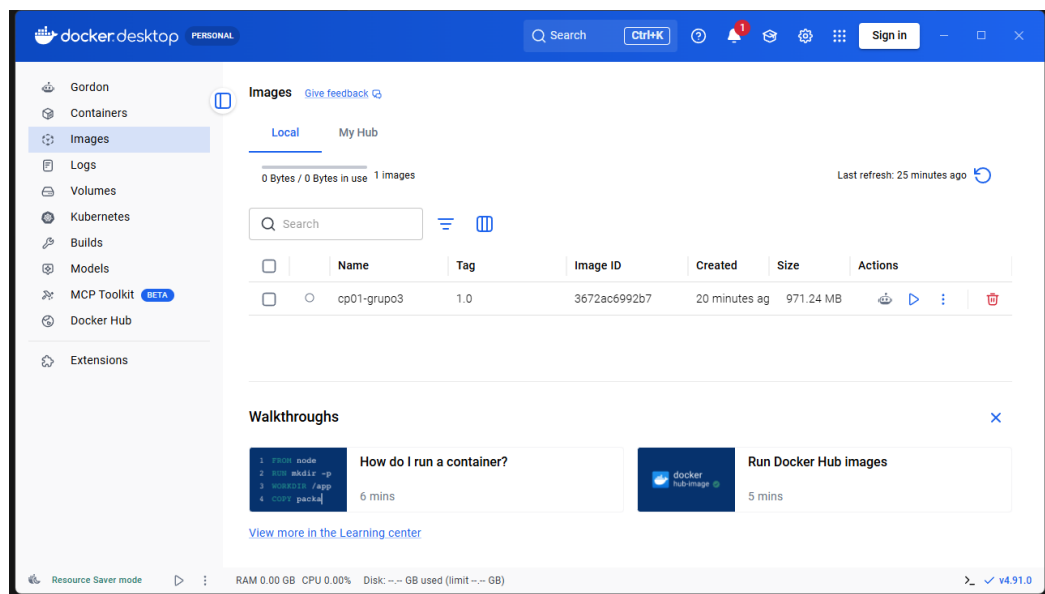


Figura 4 - Ambiente Docker e WSL 2

```
LAB GRUPO 3 - DOCKER
COMANDO: docker compose build --pull lab

Docker version 29.8.0, build 88096ef
Docker Compose version v5.5.1
WSL: Ubuntu, versão 2

Imagem: cp01-grupo3:1.0
ID: sha256:3672ac6992b77b006867348d9b05d2015cb33269a723ac86014ce11688ef4f25
Criada em: 2026-09-16T18:55:28.743786323Z
Tamanho: 971245040 bytes

RESULTADO: imagem criada com sucesso.
```

14 GATE E VERSÃO INSEGURA

O gate lê os relatórios do njsscan e do npm audit. A política usada foi:

Resultado	Decisão
SAST ERROR	Bloqueia como HIGH
SCA HIGH ou CRITICAL	Bloqueia
Sem alertas nesses níveis	Aprova

Na versão insegura, três alertas SAST e um pacote HIGH produziram código de saída 1. O status foi BLOQUEADO.

Na versão corrigida, os dois scanners terminaram com código 0. O status foi APROVADO.

O print abaixo registra o bloqueio da versão insegura. Na página seguinte, os prints mostram a aprovação após as correções. Esses resultados comprovam os testes locais no Docker.

Figura 5 - Gate bloqueando a versão insegura

```
LAB GRUPO 3 - TESTE BLOQU  X  +  v
COMANDO: docker compose run --rm lab inseguro

SAIDA REAL DO CONTAINER:
{
  "alvo": "inseguro",
  "status": "BLOQUEADO",
  "exit_code": 1,
  "politica": "SAST ERROR = HIGH por decisão do grupo; SCA HIGH/CRITICAL bloqueiam",
  "contagens": {
    "sast_high": 3,
    "sast_moderate": 0,
    "sast_low": 0,
    "sca_high": 1,
    "sca_critical": 0,
    "sca_moderate": 0,
    "sca_low": 0
  }
}

njsscan-json: 4.809s, código 1
njsscan-sarif: 4.731s, código 1
npm-audit: 0.731s, código 1

CODIGO_DE_SAIDA=1
|
```

15 VERSÃO CORRIGIDA E CONCLUSÃO

O laboratório mostrou por que SAST e SCA são diferentes. O njsscan encontrou falhas escritas no app.js. O npm audit encontrou uma versão vulnerável do lodash.

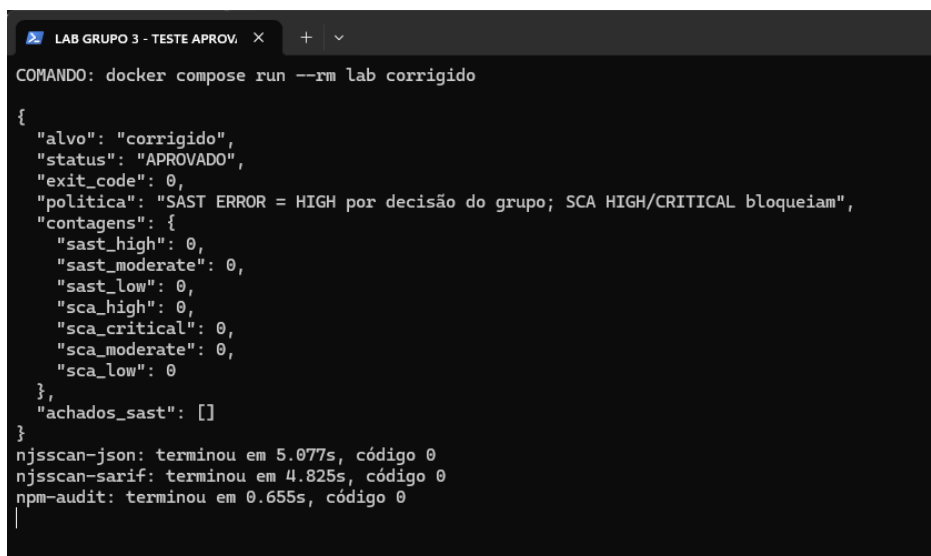
As correções também foram diferentes: retirar operações perigosas no código e atualizar a dependência. Depois disso, o gate aprovou a nova versão e o npm audit mostrou zero vulnerabilidades.

O njsscan depende das regras disponíveis; o npm audit, dos avisos conhecidos. Datree está arquivado e StackHawk possui partes comerciais.

Para um projeto pequeno, a sequência sugerida é SAST e SCA desde o começo, análise de IaC antes do deploy e DAST em um ambiente de teste autorizado.

A aprovação confirma a correção dos alertas analisados. Outros testes e a revisão do código continuam necessários.

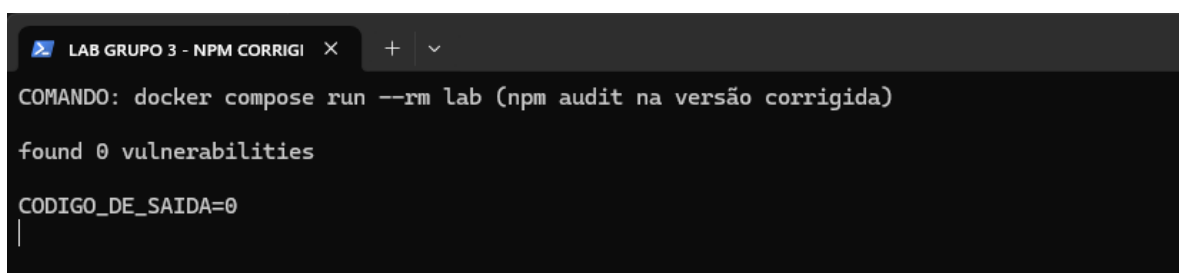
Figura 6 - Gate aprovando a versão corrigida



```
LAB GRUPO 3 - TESTE APROV. X + v
COMANDO: docker compose run --rm lab corrigido

{
  "alvo": "corrigido",
  "status": "APROVADO",
  "exit_code": 0,
  "politica": "SAST ERROR = HIGH por decisão do grupo; SCA HIGH/CRITICAL bloqueiam",
  "contagens": {
    "sast_high": 0,
    "sast_moderate": 0,
    "sast_low": 0,
    "sca_high": 0,
    "sca_critical": 0,
    "sca_moderate": 0,
    "sca_low": 0
  },
  "achados_sast": []
}
njsscan-json: terminou em 5.077s, código 0
njsscan-sarif: terminou em 4.825s, código 0
npm-audit: terminou em 0.655s, código 0
|
```

Figura 7 - npm audit corrigido



```
LAB GRUPO 3 - NPM CORRIGI X + v
COMANDO: docker compose run --rm lab (npm audit na versão corrigida)

found 0 vulnerabilities

CODIGO_DE_SAIDA=0
|
```

REFERÊNCIAS

ABRAHAM, Ajin. **njsscan**. 2026. Disponível em: <https://github.com/ajinabraham/njsscan>.

DATREE. **Datree: repositório e aviso de descontinuação**. 2023. Disponível em: <https://github.com/datreeio/datree>.

GITHUB. **SARIF support for code scanning**. s.d.. Disponível em: <https://docs.github.com/en/code-security/reference/code-scanning/sarif-files/sarif-support>.

GITHUB ADVISORY DATABASE. **Prototype Pollution in Iodash: CVE-2020-8203**. 2020. Disponível em: <https://github.com/advisories/GHSA-p6mc-m468-83gw>.

MITRE. **CWE-78: OS Command Injection**. s.d.. Disponível em: <https://cwe.mitre.org/data/definitions/78.html>.

MITRE. **CWE-79: Cross-site Scripting**. s.d.. Disponível em: <https://cwe.mitre.org/data/definitions/79.html>.

MITRE. **CWE-95: Eval Injection**. s.d.. Disponível em: <https://cwe.mitre.org/data/definitions/95.html>.

NIST. **Secure Software Development Framework (SSDF), SP 800-218**. 2022. Disponível em: <https://csrc.nist.gov/pubs/sp/800/218/final>.

NPM. **npm audit e npm SBOM**. s.d.. Disponível em: <https://docs.npmjs.com/cli/v11/commands/npm-audit/>.

STACKHAWK. **HawkScan: documentação**. s.d.. Disponível em: <https://docs.stackhawk.com/hawkscan/>.

ANEXO A. USO DE IA

A inteligência artificial foi usada apenas para a revisão do trabalho e para a organização dos slides.